Sri Lanka Institute of Information Technology

SE2020 – Web and Mobile Technologies
Year 2, Semester 2- 2026

Practical Answer Submission

Practical Sheet No: 09

Student ID	IT24102099
Student Name	Mathuppriya Naguleswaran
Campus/ Center name	SLIIT – Northen UNI
Specialization	Software Engineering
Batch No	1

MongoDB

1. Initialize Node project.

npm init -y

- This creates a package.json file.

```
PS C:\Users\vallu\Desktop\campus-food-api_It24102099> npm init -y
Wrote to C:\Users\vallu\Desktop\campus-food-api_It24102099\package.json:

{
  "name": "campus-food-api_it24102099",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Figure 9.1: Initializing the Node.js project and creating the package.json file using npm init -y

2. Install required packages

npm install express mongoose cors dotenv

- express → to create the API server
- mongoose → to connect Node.js with MongoDB
- cors → to allow cross-origin requests
- dotenv → to load values from .env

```
PS C:\Users\vallu\Desktop\campus-food-api_It24102099> npm install express mongoose cors dotenv

added 85 packages, and audited 86 packages in 14s

25 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\vallu\Desktop\campus-food-api_It24102099>
```

Figure 9.2: Installing required dependencies (express, mongoose, cors, dotenv) using npm

3. Setup MongoDB connection

For the data base im using local MongoDB. So I don't have any password for that. If I use MongoDB Atlas **must** have database username , database password, cluster connection string.

In .env file,

PORT=3000

MONGO_URI=mongodb://127.0.0.1:27017/campus_food

- localhost = my own computer
- 27017 = default MongoDB port
- campus_food = database name, MongoDB will create it when data is inserted

```
npm start\Users\vallu\Desktop\campus-food-api_It24102099>

> campus-food-api_it24102099@1.0.0 start
> node server.js

[dotenv@17.3.1] injecting env (2) from .env -- tip: ⚙ suppress all logs with { quiet: true }
✅ Connected to MongoDB
🚀 Server running on http://localhost:3000
```

Figure 9.3: Successful server startup and MongoDB connection using npm start

Add start script in package.json

When open the package.json its have this version of code

```
package.json > ...
{
  "name": "campus-food-api_it24102099",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  ▷Debug
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "cors": "^2.8.6",
    "dotenv": "^17.3.1",
    "express": "^5.2.1",
    "mongoose": "^9.3.3"
  }
}
```

```
package.json > ...
{
  "name": "campus-food-api_it24102099",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  ▷Debug
  "scripts": {
    "start": "node server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "cors": "^2.8.6",
    "dotenv": "^17.3.1",
    "express": "^5.2.1",
    "mongoose": "^9.3.3"
  }
}
```

But here I want to change and add extra things related to my project

My project have server.js NOT index.js

And add **"start": "node server.js"**

- This tells npm what to run when you type: npm start

4. Run the server

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\vallu\Desktop\campus-food-api_It24102099> npm start

> campus-food-api_it24102099@1.0.0 start
> node server.js

[dotenv@17.3.1] injecting env (2) from .env -- tip: 🔐 prevent building .env in docker: https://dotenvx.com/prebuild
✅ Connected to MongoDB
🚀 Server running on http://localhost:3000
```

Figure 9.4: Successful execution of the Node.js server with MongoDB connection established

5. Test in Postman

1. For add a student details.

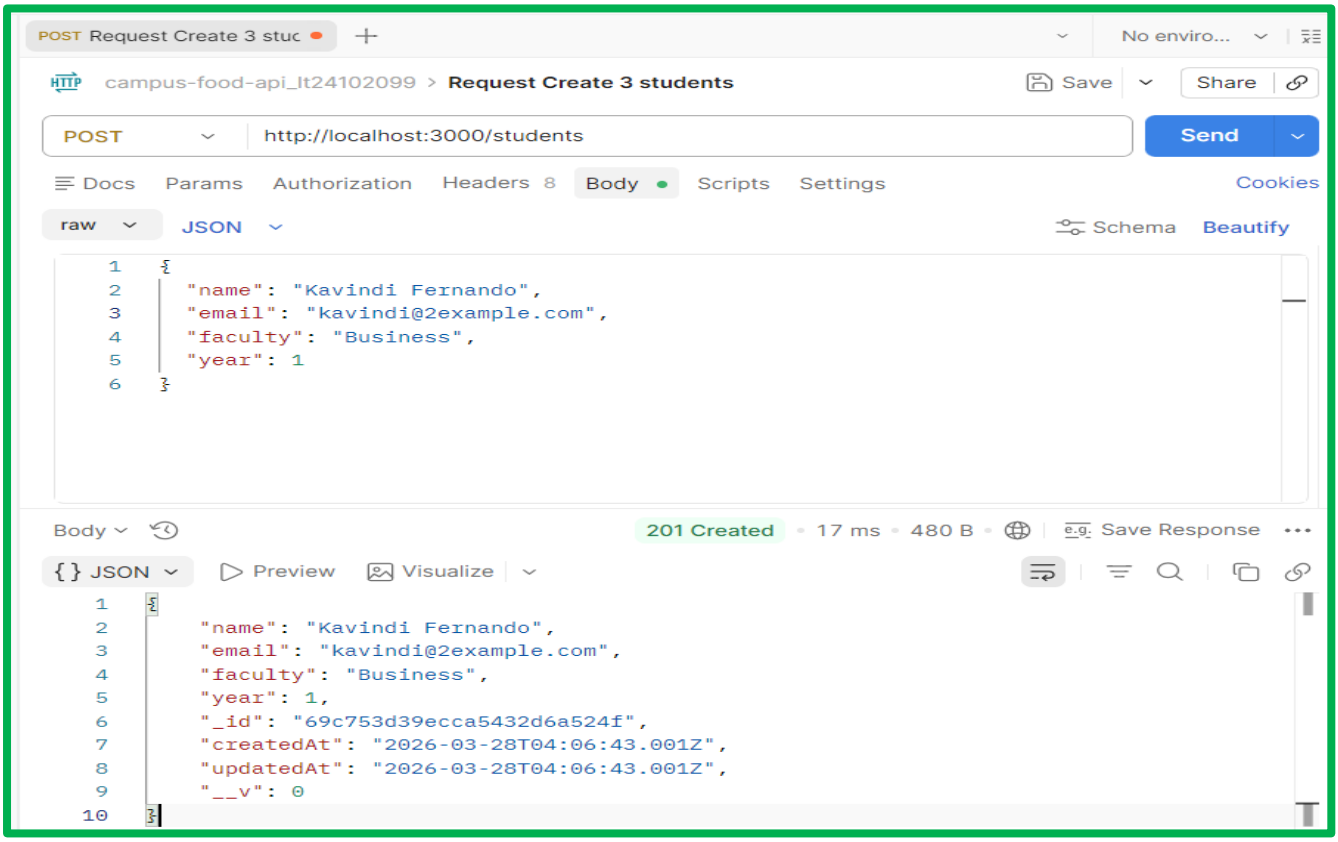

Figure 9.5: Creating a new student using POST /students endpoint in Postman

2. Get a student by ID

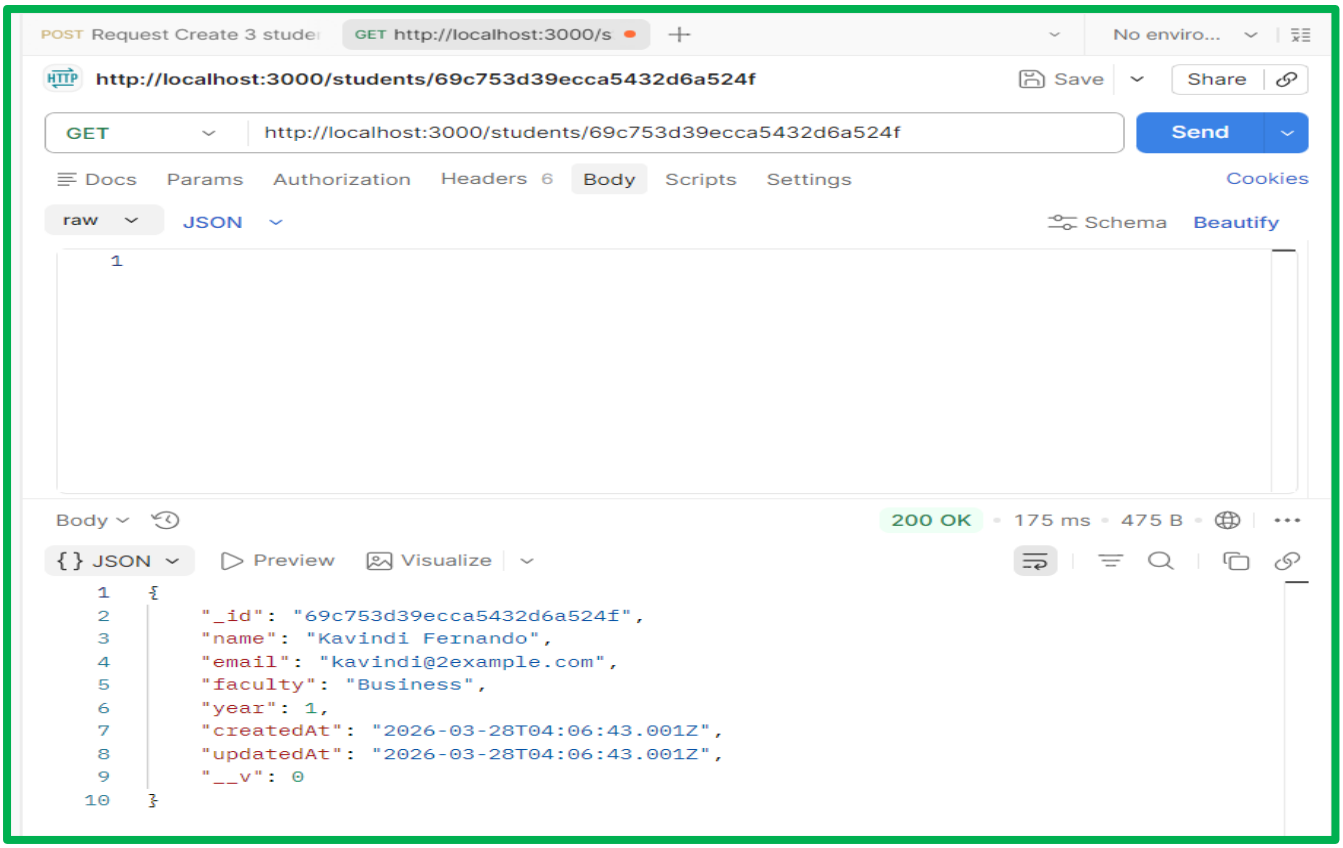

Figure 9.6: Retrieving a student by ID using GET /students/:id endpoint in Postman

3. Add a menu item

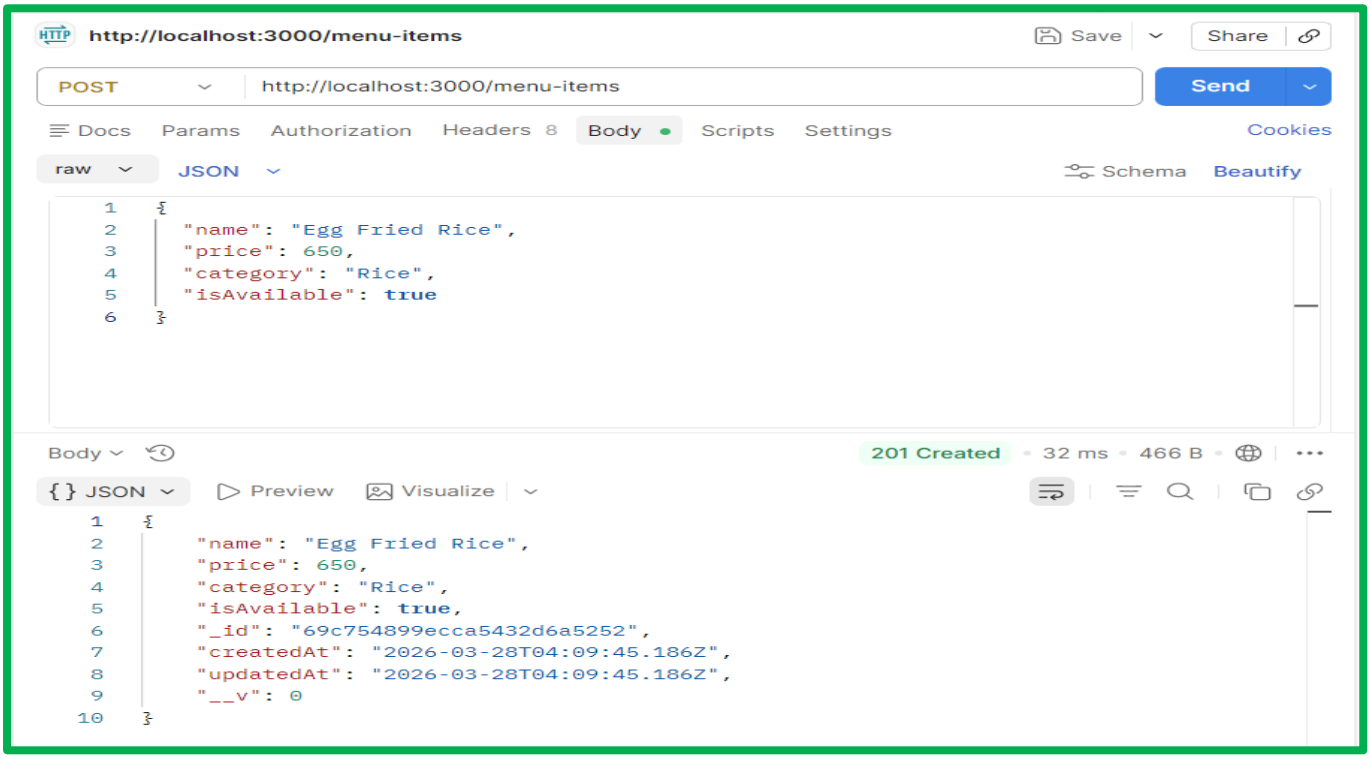

Figure 9.7: Creating a new menu item using POST /menu-items endpoint in Postman

4. List menu items

POST Request Create 3 stude | GET Get a student by ID | GET Create 5 menu items | No enviro...

campus-food-api_It24102099 > **Create 5 menu items** Save Share

GET http://localhost:3000/menu-items Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Body 200 OK 90 ms 1.2 KB Save Response

{} JSON Preview Visualize


```
[
    {
        "_id": "69c7552e9ecca5432d6a525a",
        "name": "Tea",
        "price": 80,
        "category": "Beverage",
        "isAvailable": true,
        "createdAt": "2026-03-28T04:12:30.123Z",
        "updatedAt": "2026-03-28T04:12:30.123Z",
        "__v": 0
    },
    {
        "_id": "69c7550b9ecca5432d6a5258",
        "name": "Fish Bun",
        "price": 100,
        "category": "Snack",
        "isAvailable": true,
        "createdAt": "2026-03-28T04:11:55.025Z",
        "updatedAt": "2026-03-28T04:11:55.025Z",
        "__v": 0
    },
    {
        "_id": "69c754da9ecca5432d6a5256",
        "name": "Coke",
        "price": 180,
        "category": "Beverage",
```


Figure 9.8: Retrieving all menu items using GET /menu-items endpoint in Postman.

5. Search menu items

By name:

http://localhost:3000/menu-items/search Save Share

GET http://localhost:3000/menu-items/search?name=egg Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
name	egg		
Key	Value	Description	

Body 200 OK 50 ms 463 B

{} JSON Preview Visualize


```
[
    {
        "_id": "69c754899ecca5432d6a5252",
        "name": "Egg Fried Rice",
        "price": 650,
        "category": "Rice",
        "isAvailable": true,
        "createdAt": "2026-03-28T04:09:45.186Z",
        "updatedAt": "2026-03-28T04:09:45.186Z",
        "__v": 0
    }
]
```


Figure 9.9: Searching menu items by name using GET /menu-items/search with query parameter

By category:

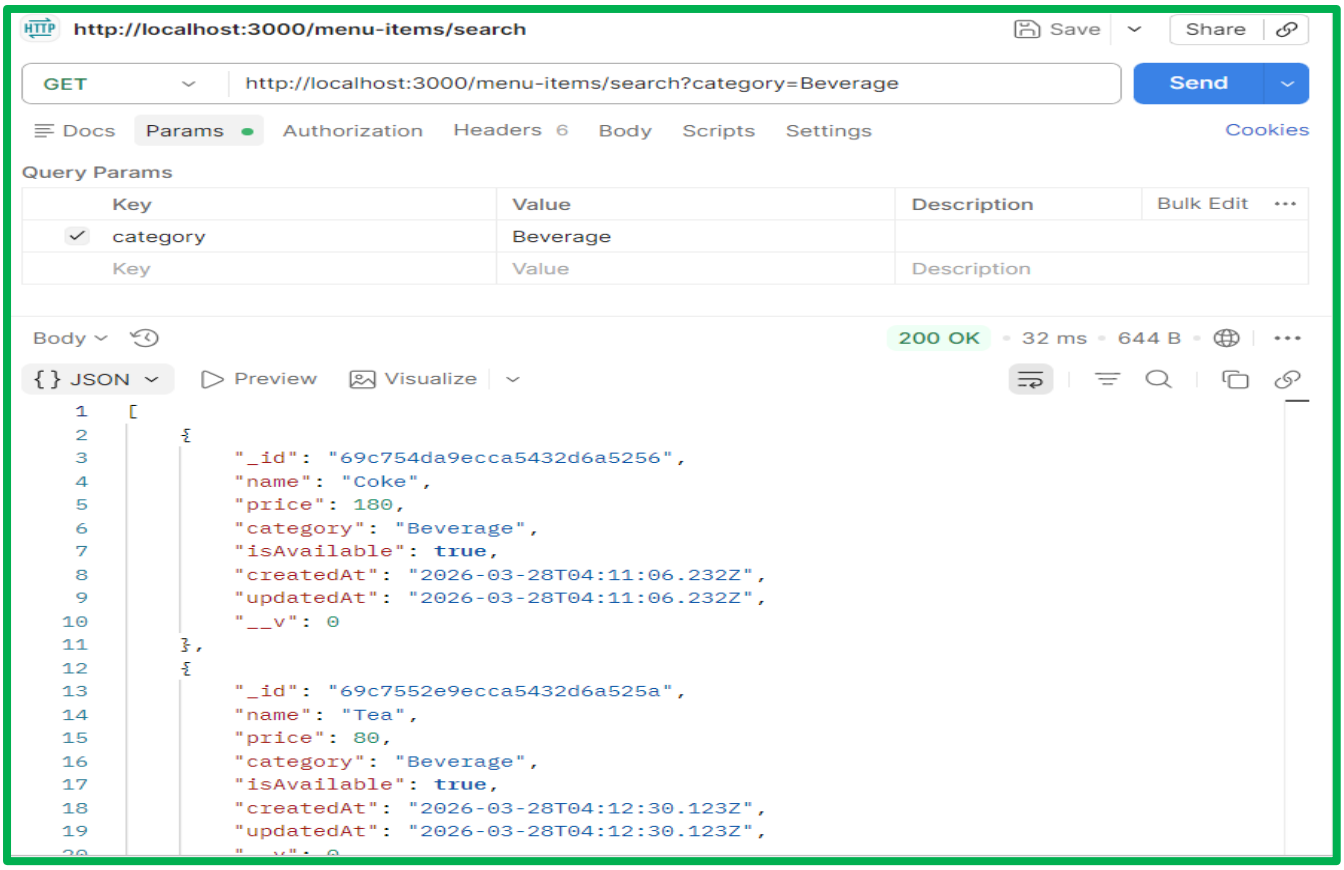

Figure 9.10: Searching menu items by category using GET /menu-items/search with query parameter

By both:

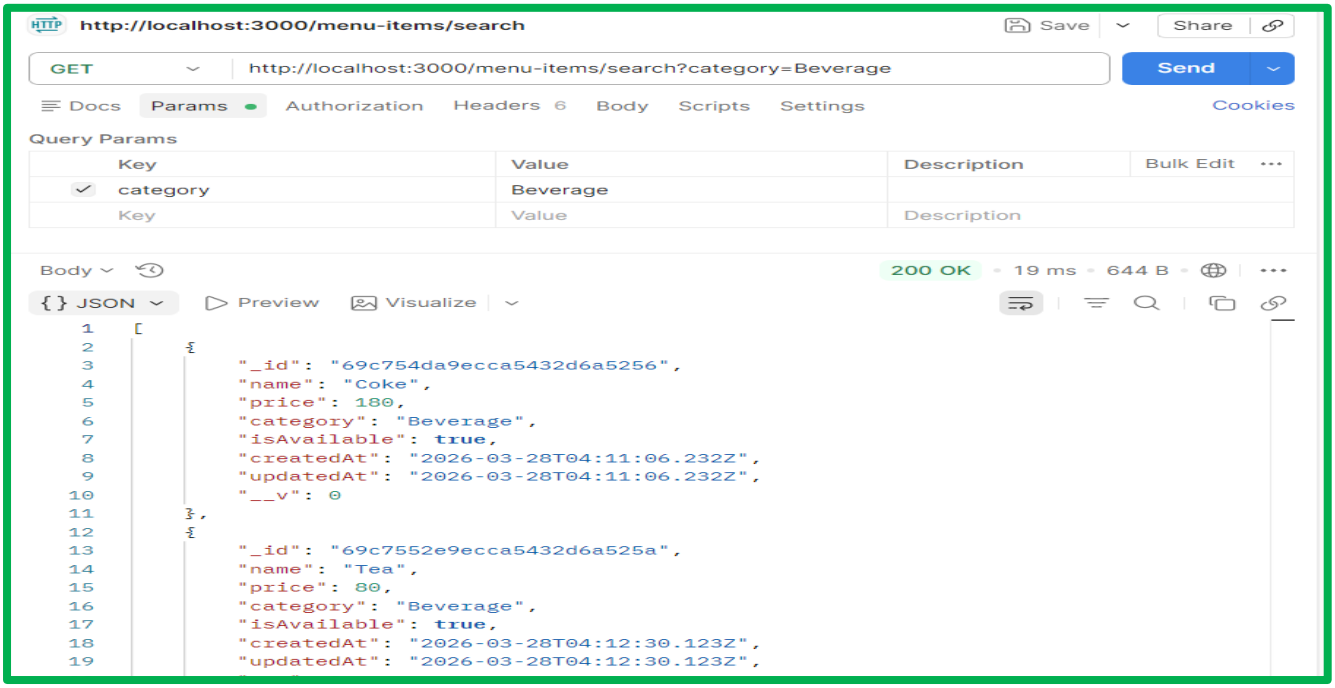

Figure 9.11: Output of category-based filtering showing multiple matching menu items retrieved from MongoDB

6. Place an order

http://localhost:3000/orders

POST http://localhost:3000/orders

Docs Params Authorization Headers 8 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON


```
{
  "student": "69c753d39ecca5432d6a524f",
  "items": [
    {
      "menuItem": "69c7550b9ecca5432d6a5258",
      "quantity": 2
    },
    {
      "menuItem": "69c754da9ecca5432d6a5256",
      "quantity": 1
    }
  ]
}
```


Figure 9.12: Placing a new order using POST /orders endpoint with student and menu item references

7. Paginated Orders

http://localhost:3000/orders Save Share

GET http://localhost:3000/orders?page=1&limit=2 Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
page	1		
limit	2		
Key	Value	Description	

Body 200 OK 107 ms 1.08 KB

{ } JSON Preview Visualize


```
{
    "page": 1,
    "limit": 2,
    "totalOrders": 1,
    "totalPages": 1,
    "orders": [
        {
            "_id": "69c758459ecca5432d6a5265",
            "student": {
                "_id": "69c753d39ecca5432d6a524f",
                "name": "Kavindi Fernando",
                "email": "kavindi@2example.com",
                "faculty": "Business",
                "year": 1,
                "createdAt": "2026-03-28T04:06:43.001Z",
                "updatedAt": "2026-03-28T04:06:43.001Z",
                "__v": 0
            },
            "items": [
                {
```


Figure 9.13: Execution of paginated GET /orders API request demonstrating retrieval of order data with metadata and populated student and item details

8. Get order by ID

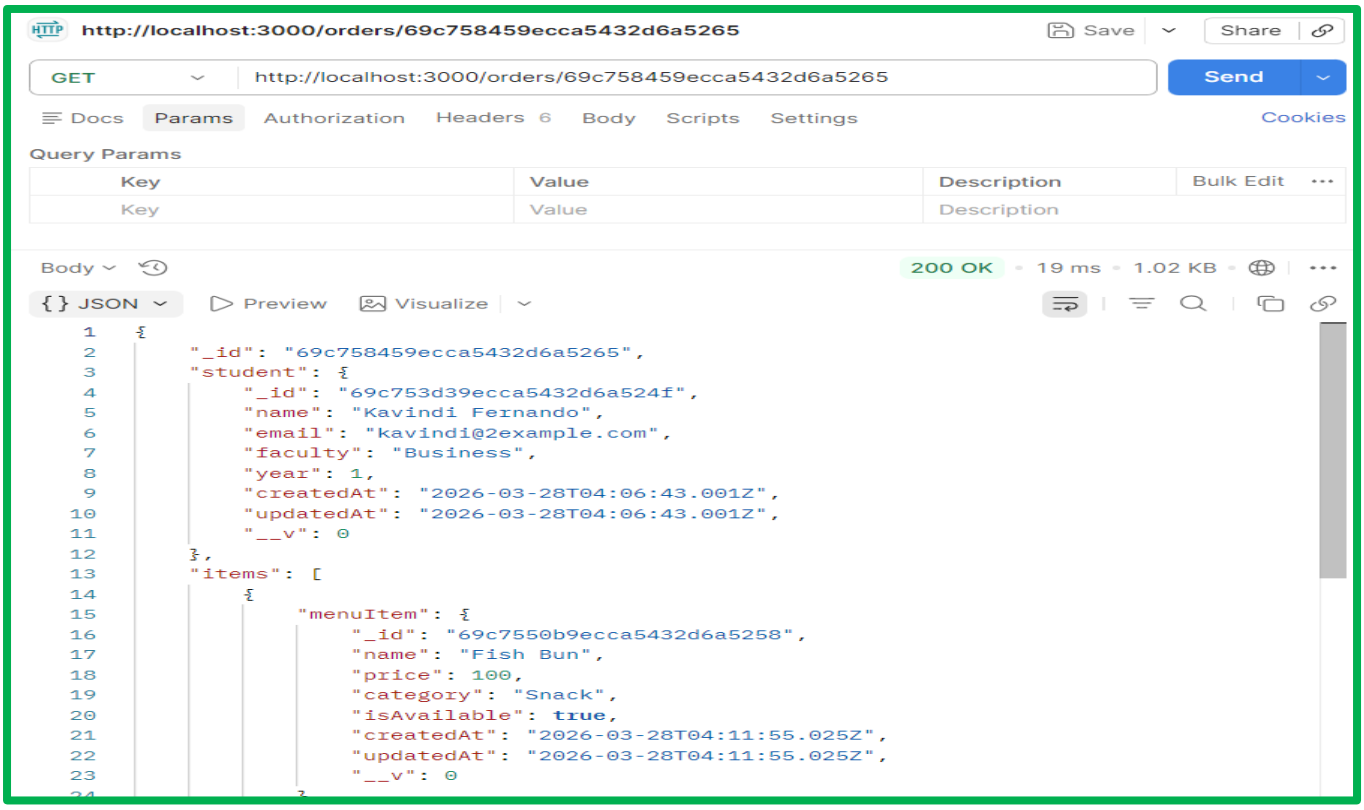

Figure 9.14: Retrieving a specific order by ID using GET /orders/:id endpoint

9. Update order status

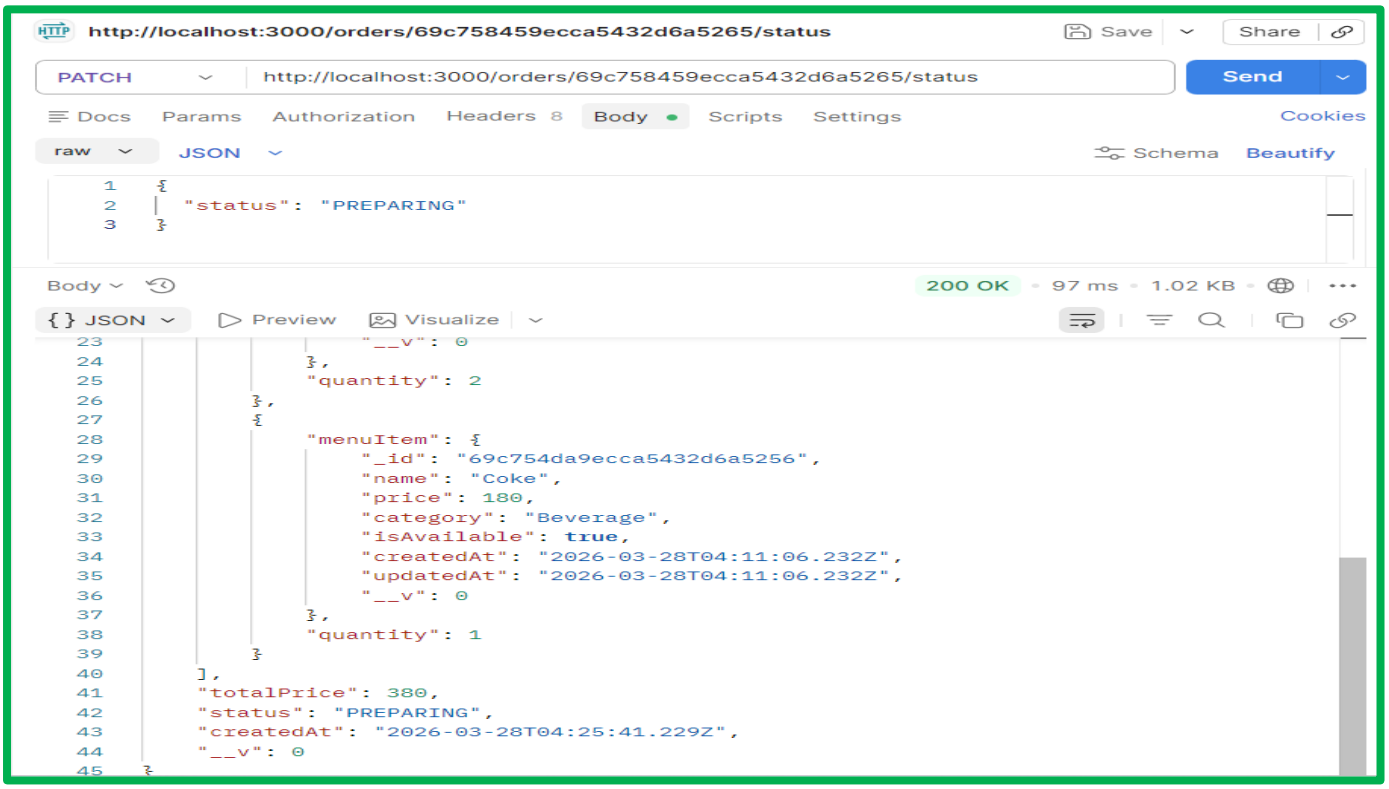

Figure 9.15: Execution of PATCH /orders/:id/status API endpoint demonstrating modification of order status with updated response from MongoDB

10. Delete order

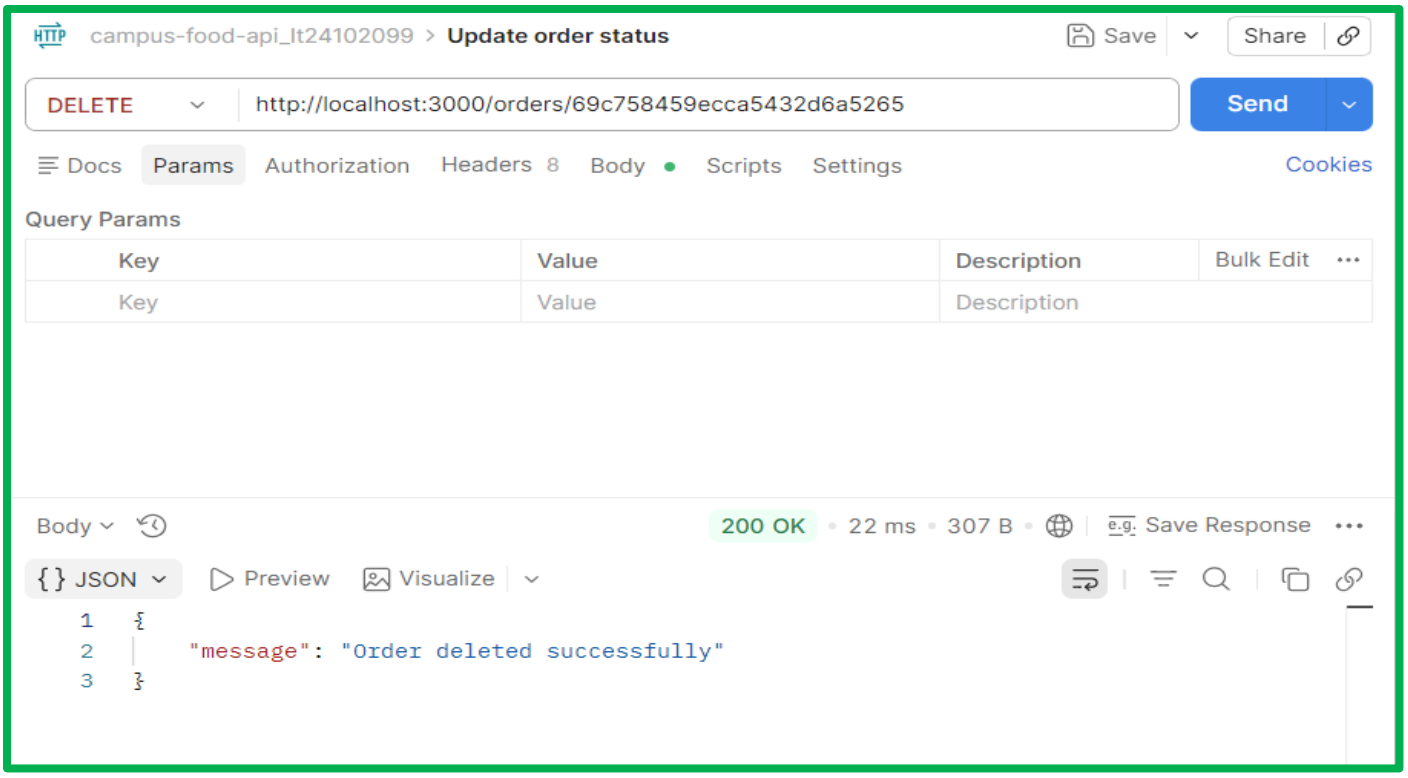

Figure 9.16: Deleting an order using DELETE /orders/:id endpoint

11. Analytics & Aggregation Endpoints

1. Total Amount Spent by a Student

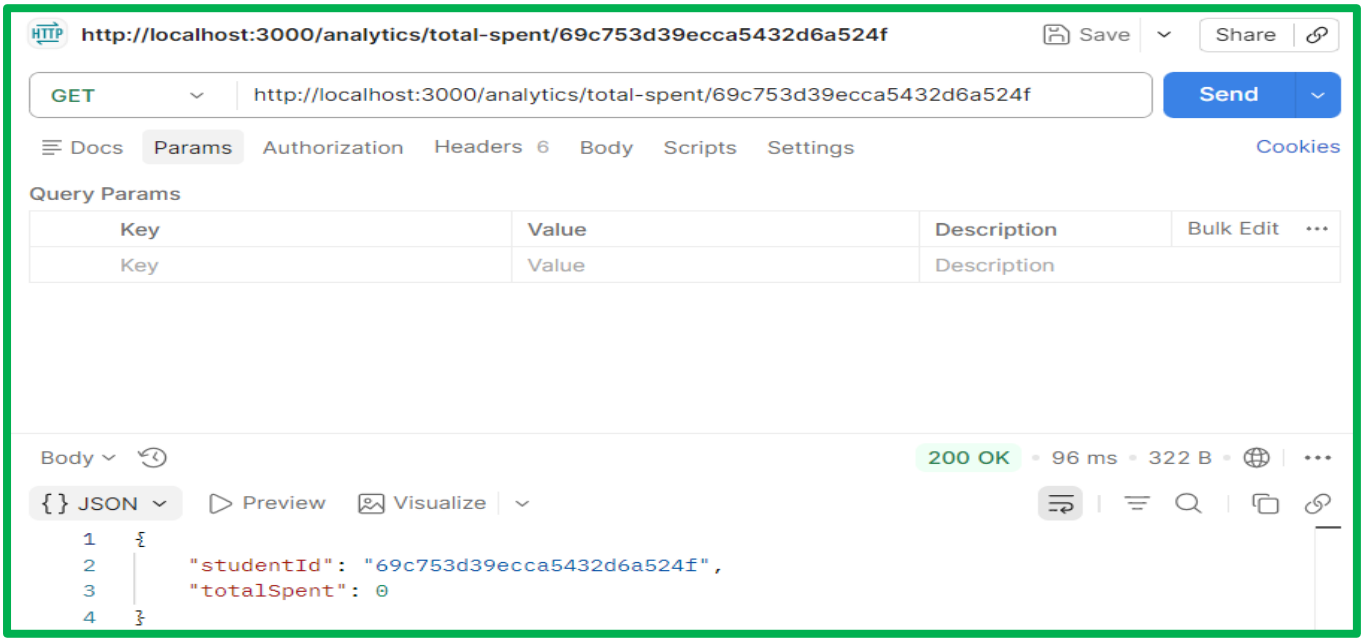

Figure 9.17: Calculating total amount spent by a student using GET /analytics/total-spent/:studentId endpoint

2. Top Selling Menu Items

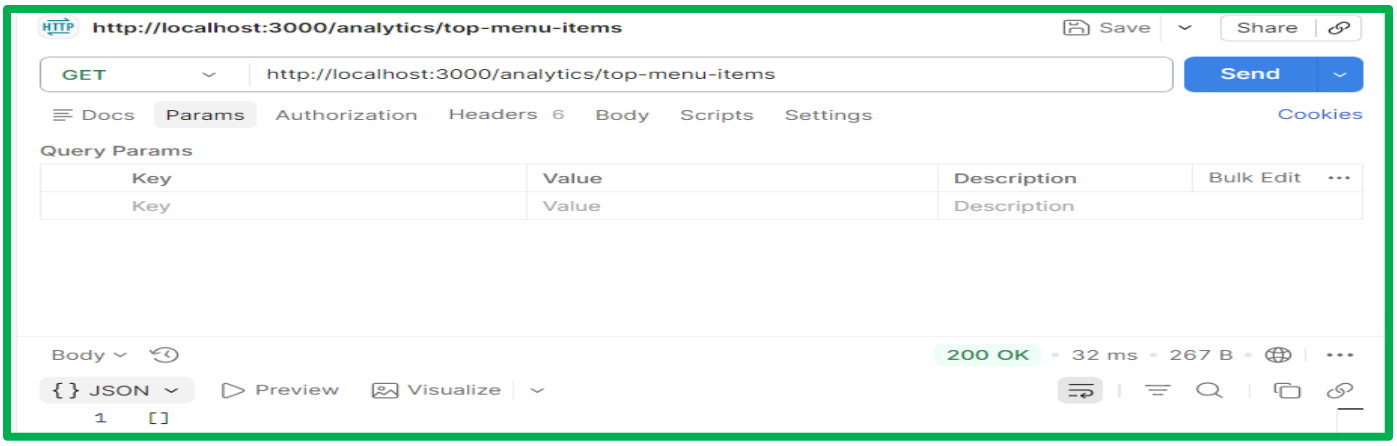

Figure 9.18: Retrieving top selling menu items using GET /analytics/top-menu-items endpoint

3. Top menu items with limit

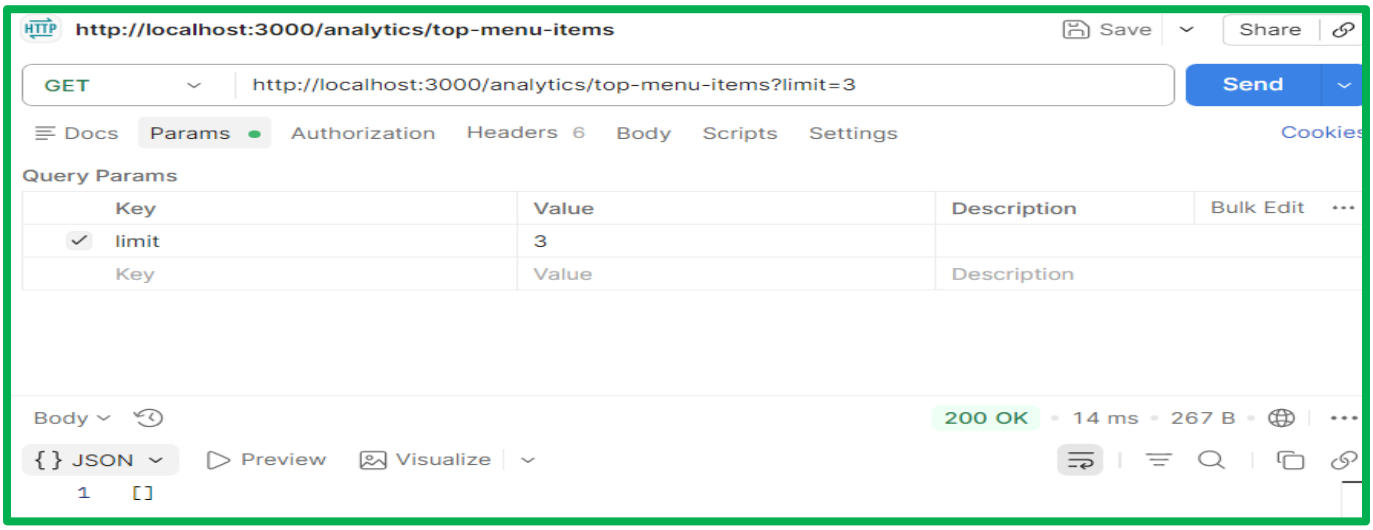

Figure 9.19: Retrieving top menu items with limit using GET /analytics/top-menu-items?limit=3 endpoint

4. Daily Order Counts

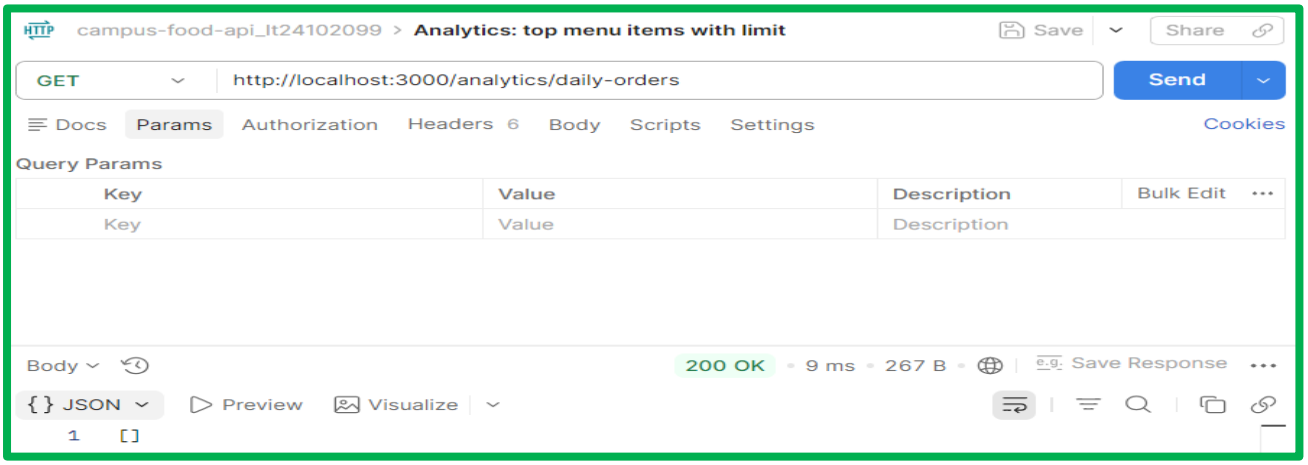

Figure 9.20: Retrieving daily order counts using GET /analytics/daily-orders endpoint